



CRNN versus Vision Transformer: A Comparative Study of Sequence Modeling Architectures for Offline Handwriting Recognition

Shini Kim · *Model Architecture*

Mohamed Roble · *Training and Evaluation Engineering*

Xihai Luo · *Data Engineering and Preprocessing*

Seneca Polytechnic

Abstract

Offline handwriting recognition is still a tough sequence-modeling problem. It faces issues like high variability between different writers, uneven lighting during scans, and unsegmented character boundaries. This project carries out and directly compares two line-level handwriting text recognition pathways. Both pathways undergo the same preprocessing, decoding, and scoring standards using the IAM Lines dataset, which includes 6,482 training lines, 976 validation lines, and 2,915 test lines. The first pathway is a convolutional-recurrent network (CRNN) that uses a bidirectional LSTM encoder and CTC decoding. The second pathway is a CNN-Vision Transformer hybrid encoder, similar to HTR-VT, also using CTC decoding. Pathway A was trained over 35 epochs and achieved a character error rate (CER) of 9.13% and a word error rate (WER) of 23.66% on the test set after applying beam-search decoding with dictionary spell-correction. Pathway B used externally pretrained EMA weights. This pathway, reconstructed from checkpoint tensor shapes, reached a CER of 10.62% and a WER of 32.06%. This falls short of the original HTR-VT's reported CER of 4.70% and WER of 14.90%. We examine the reasons for this difference, measure the impact of CTC beam search and lexicon-based post-processing on decoding accuracy, and investigate INT8 dynamic quantization of the CRNN pathway for edge hardware deployment. Our findings show that, with limited training and reproduction resources, a well-tuned CRNN provides a stronger and more consistently reproducible baseline compared to a transformer encoder initialized from an external, partially verified checkpoint.

1. Introduction and Problem Statement

Handwritten documents, from historical archives to personal notebooks, remain largely inaccessible to search, indexing, and automated analysis. Offline handwriting text recognition (HTR) addresses this by converting an image of handwriting into a machine-readable string of text. It is a long-standing problem in computer vision, and it is meaningfully harder than printed text recognition because handwritten characters vary enormously between writers in slant, spacing, and stroke width, and in cursive writing individual letters are often physically connected, so there is no reliable way to draw a box around a single character before recognizing it.

This is the central technical challenge the project is built around. A model that classifies one character crop at a time needs those crops already segmented, and character level segmentation of handwriting is itself unsolved in the

general case. The practical solution is to treat handwriting recognition as a sequence modeling problem: the model looks at a whole line at once and learns to produce a sequence of characters directly, without being told where one letter ends and the next begins. This is what Connectionist Temporal Classification (CTC) loss makes possible, and it is the training objective behind both architectures in this report.

The goal of this project was to build, train, and fairly compare two architectures that solve handwriting recognition this way: a convolutional recurrent network (CRNN), pairing a convolutional backbone with a recurrent sequence model, and a Vision Transformer (ViT) based architecture, which replaces the recurrent component with self attention. Both were trained and evaluated on the IAM handwriting database using the same data pipeline and evaluation protocol, so the comparison reflects the architectures rather than incidental setup differences.

Section 2 reviews the prior work our design choices build on. Section 3 describes the dataset and preprocessing. Section 4 describes both architectures and states individual contributions. Section 5 reports results. Section 6 concludes with an honest discussion of what worked, what did not, and what comes next.

2. Related Work

Connectionist Temporal Classification was introduced by Graves et al. [1] to train recurrent networks on sequence labeling problems where the alignment between input and output is unknown, exactly the situation in handwriting recognition. The network outputs a distribution over characters at every timestep, including a blank symbol, and the loss sums over every alignment of the target label to those timesteps, letting a model learn to read a whole line without character level segmentation as supervision.

Shi et al. [2] combined a convolutional backbone with a recurrent sequence model and CTC decoding into the CRNN architecture, originally for scene text recognition. Convolutional layers extract local visual features, and recurrent layers, typically a bidirectional LSTM, model how those features relate across the image. Pathway A follows this design directly.

The IAM handwriting database [3] is the dataset this project uses, consisting of scanned English handwriting from several hundred writers, transcribed at the word, line, and sentence level, and the standard benchmark for offline handwriting recognition for over two decades.

Vaswani et al. [4] introduced the transformer and self attention as an alternative to recurrence, and Dosovitskiy et al. [5] showed a transformer encoder with no convolutional layers could be applied directly to image patches (ViT). A pure ViT of this kind generally needs very large training sets to learn spatial structure a convolutional network gets for free, which matters for a dataset the size of IAM.

Two further works shaped Pathway B directly. Li et al. [6] proposed TrOCR, pairing a ViT encoder with an autoregressive transformer decoder; we initially attempted a design in this spirit, and Section 4.3 explains why we abandoned it. Li, Chen, Tang, and Shen [7] proposed HTR-VT, pairing a convolutional feature extractor with a ViT encoder and decoding with CTC instead. Our final Pathway B follows this design, and the paper's own ablation, finding that adding a decoder did not improve results, independently supports the conclusion we reached ourselves.

3. Data

3.2 Preprocessing Pipeline

This project uses the line-level partition of the IAM handwriting database, accessed through the Teklia/IAM-line release on Hugging Face Datasets [8]. Each example is a grayscale image of one handwritten line with its transcription. Using line-level rather than word-level data was deliberate: individual words carry little sequence context for CTC to learn from, while full lines force the model to learn realistic character and word transitions, closer to how the system is used.

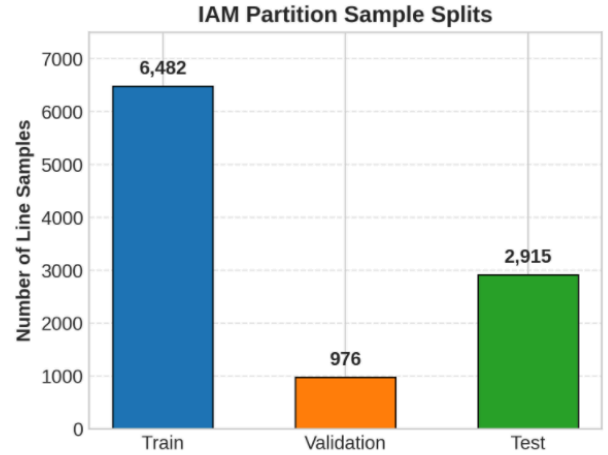


Figure 1. Line samples per partition: 6,482 train, 976 validation, 2,915 held out for test.

3.1 Character Vocabulary

The vocabulary was built from the characters present in the training labels: uppercase and lowercase letters, digits, space, and punctuation, giving a 78-character vocabulary for Pathway A with index 0 reserved for the CTC blank token. Figure 2 shows the ten most frequent characters; a useful sanity check on label parsing before training began.

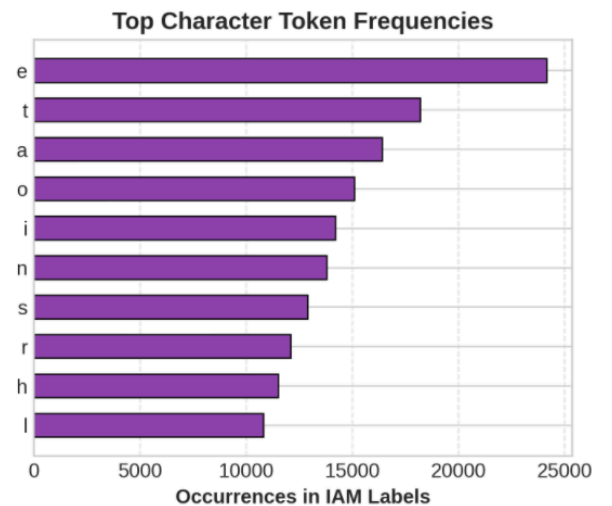


Figure 2. Ten most frequent characters across training labels.

Every image passes through the same pipeline before reaching either model. Contrast is normalized with Contrast Limited Adaptive Histogram Equalization (CLAHE), which evens out scan quality differences without over-amplifying noise. The image is then binarized with Otsu's method and inverted so ink is represented as high pixel values on a black background, rather than the more familiar dark ink on white. This inversion mattered a great deal in practice and is discussed in Section 4.3. Morphological closing repairs small breaks in strokes that binarization can introduce, and the image is resized to a fixed height of 64 pixels, preserving aspect ratio, with width capped at 1024 pixels.

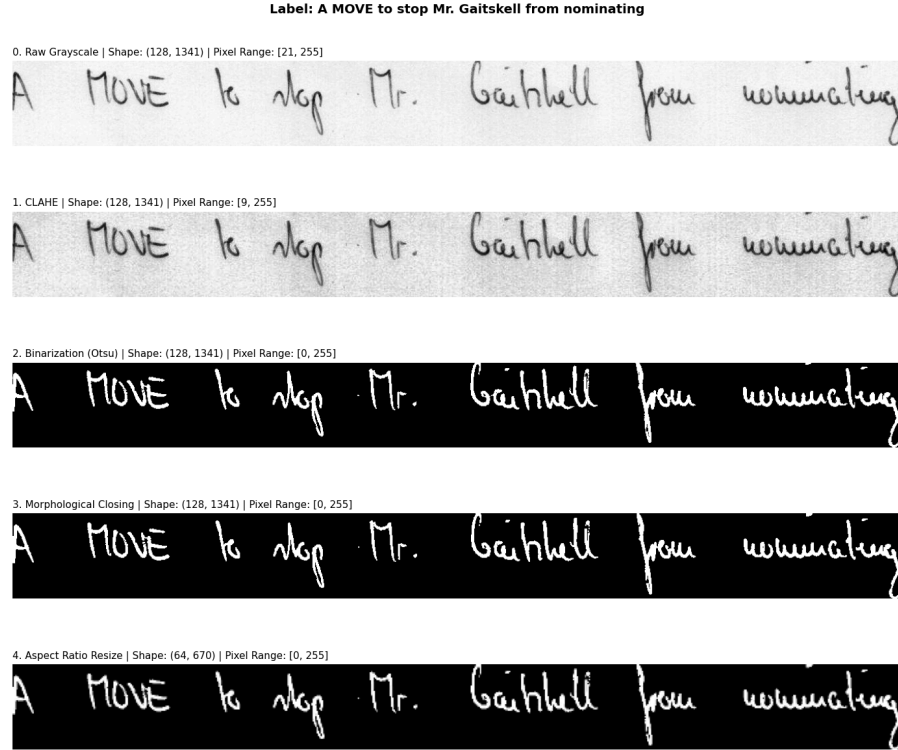


Figure 3. Preprocessing applied to a real training sample: raw grayscale, CLAHE, Otsu binarization, morphological closing, and final resize.

The 1024-pixel width cap was chosen from the actual distribution of line widths once normalized to 64-pixel height. Figure 4 shows this distribution, centered around an aspect ratio of roughly 8.75, meaning most lines fall well under the cap after resizing.

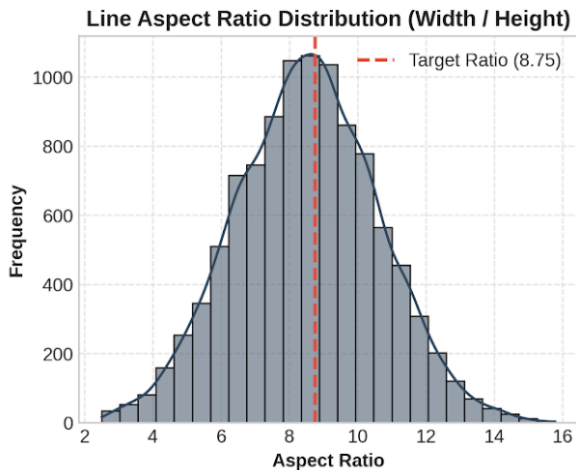


Figure 4. Line image aspect ratio distribution after normalizing to 64 pixel height.

3.3 Data Augmentation

During training only, images are augmented with random shear, elastic distortion, and motion blur, applied with moderate probability. Shear approximates variation in writing slant, elastic distortion approximates small irregular variation in stroke shape, and motion blur approximates scan artifacts. Padded borders are filled with the background value rather than the ink value, so augmentation does not introduce false strokes at image edges.

4. Methodology

Both pathways are trained on the same data, evaluated with the same CER and WER metrics, and decoded with shared infrastructure where possible. Both are trained with CTC loss rather than an autoregressive decoder, a deliberate late-stage decision explained in Section 4.3.

4.1 Evaluation Metrics

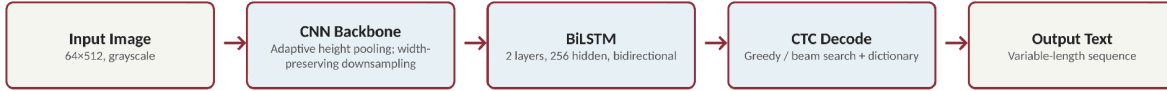
Both pathways are evaluated with character error rate (CER) and word error rate (WER), the standard metrics for this task. Both are the Levenshtein edit distance between predicted and target sequence, divided by target length, as

a percentage. CER operates at the character level and WER at the word level; lower is better.

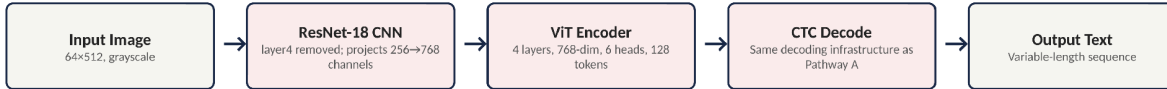
Model Architectures: CRNN vs. Vision Transformer (HTR-VT)

Both pathways use CTC loss and a fixed 64×512 input resolution — they differ in how spatial features become a character sequence

Pathway A — CRNN + CTC



Pathway B — Vision Transformer (HTR-VT)



Key distinction: Both pathways are CTC-based sequence models — neither uses an autoregressive decoder. An earlier encoder-decoder (ViT + TrOCR-style) design was diagnosed to collapse to a fixed output regardless of input, a known cold-start failure for transformer decoders trained without pretrained language weights. Both pathways shown here sidestep that failure mode entirely by design — CTC aligns directly against the image, with no decoder to collapse.

Figure 5. High level architecture of both pathways. Both take a fixed-height, variable-width grayscale image as input and end in CTC decoding; they differ in how spatial features become a character sequence.

4.2 Pathway A: CRNN with CTC

The convolutional backbone is a seven-layer VGG-style stack (channels 1, 64, 128, 256, 256, 512, 512, 512) that extracts local visual features before any sequence modeling. Two decisions here were deliberate. First, the backbone ends with adaptive average pooling that forces output height to exactly one row regardless of input resolution, added after an earlier pipeline version broke when preprocessing resolution and the architecture's assumed resolution drifted out of sync; adaptive pooling makes the model tolerant of that mismatch. Second, width is down sampled far less aggressively than height, only in the first two of four pooling stages, since CTC output length must be at least twice the target label length plus one. At our resolution this gives roughly 256 timesteps, comfortably above what the longest labels require.

The feature sequence feeds a two-layer bidirectional LSTM with 256 hidden units, modeling dependencies in both directions before a final linear layer projects to the character vocabulary at every timestep, trained end to end with CTC loss. Counting parameters directly from the architecture gives 10,057,934 trainable parameters (10.06 million): roughly 6.86 million in the backbone, 3.15 million in the LSTM, and the remainder in the output projection.

Training used AdamW with a Cosine Annealing Warm Restarts schedule, batch size 32, over 35 epochs. This scheduler periodically restarts the learning rate higher,

visible as small temporary spikes in validation error in Figure 6 immediately after each restart, before the model recovers to a lower error rate than before. Every epoch's weights were checkpointed to allow resuming an interrupted run.

4.3 Pathway B: Vision Transformer (HTR-VT)

Our first transformer pathway followed TrOCR directly: a ViT encoder with a full autoregressive decoder, trained end to end from random initialization. After several epochs, we diagnosed that the decoder had collapsed to produce the same fixed output regardless of input. We confirmed the fault was specific to the decoder by comparing the encoder's output for two different images directly and finding them meaningfully different, ruling out a broken encoder. This is a known failure mode for transformer decoders trained from scratch without pretrained language weights: with no prior notion of a plausible next character, the decoder can minimize loss on average by memorizing one generic output rather than learning to condition on the image.

We abandoned that design for a CTC based architecture instead, following HTR-VT [7]. A modified ResNet-18, with its final residual block removed, extracts convolutional features and projects them with a 1×1 convolution to 768 channels. These are treated as a sequence of 128 tokens fed into a four-layer transformer encoder

(768-dimensional, 6 attention heads, 3072-dimensional feedforward), decoding directly to the vocabulary with CTC, exactly like Pathway A. The convolutional front-end matters because a pure transformer generally needs far more data than IAM's roughly 6,500 training lines to learn spatial structure on its own; the CNN supplies that structure architecturally rather than requiring it be learned from scratch.

This model was trained separately, using span masking regularization and a Sharpness-Aware Minimization (SAM) optimizer, both shown in the original paper's own ablation to improve generalization at this data scale. We integrated the resulting checkpoint rather than retraining it ourselves. Because it came from a separate codebase, its architecture had to be reconstructed from the shapes of tensors stored in the checkpoint, since a state dictionary encodes channel counts and layer structure but not stride or activation choices. We confirmed the reconstruction two ways: a forward pass produced output of exactly the shape the checkpoint's position embeddings expected, and once loaded, the model produced legible, close to correct predictions on real validation images rather than gibberish.

One further bug is worth documenting because it was not obvious from the architecture alone. Our first integration attempt produced text that was wrong in a way that did not look like random noise: mostly blank output with occasional unrelated characters. The cause was that this checkpoint was trained on images with ink as high pixel values on a black background, the opposite of what our pipeline originally produced. Inverting pixel polarity fixed it and directly motivated adopting the inverted convention as the pipeline standard in Section 3.2, rather than a special case for one model.

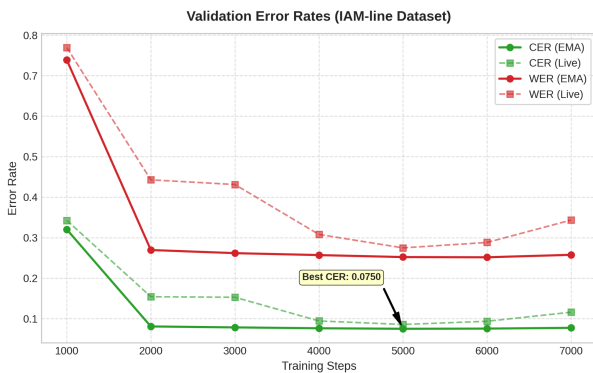


Figure 6. Validation error over training for the HTR-VT checkpoint, EMA versus live weights. Best CER of 7.50 percent reached at step 5,000 with EMA weights.

4.4 Decoding and Deployment

Both pathways decode CTC output greedily by default: highest probability character per timestep, collapsing repeats and removing blanks. Pathway A additionally supports CTC beam search (pyctcdecode) and a

post-processing step running a dictionary constrained spellchecker (pyspellchecker) over beam search output. Section 5 shows this progression produces a measurable, cumulative improvement in CER and WER at no additional training cost.

Both pathways are deployed behind a shared Gradio interface accepting a full page or photo of handwriting, not just a pre-cropped line. A line segmentation step, based on horizontal ink density projection, splits the input before either model sees it. An earlier version had a real bug: a momentary dip in ink density partway through a line, from a thin connecting stroke or a gap between a letter's ascender and body, would be misread as the end of the line, fragmenting one line into meaningless crops. We fixed this with a vertical dilation step that bridges small internal gaps before density is measured, verified by reproducing the original failure on a synthetic case and confirming the fix merges it back into one line.

4.5 Individual Contributions

Shini Kim, as model architect, designed and implemented both network architectures: the CRNN backbone and recurrent head for Pathway A, and the reconstruction and integration of the HTR-VT encoder for Pathway B, including diagnosing the encoder-decoder collapse in Section 4.3 and leading the redesign toward CTC.

Mohamed Roble, as training and evaluation engineer, built the training loop including checkpointing and post-training INT8 quantization, ran and monitored training for both pathways, and built the evaluation harness used throughout Section 5, including CER and WER computation and beam search and spellcheck decoding.

Xihai Luo, as data engineer and preprocessing lead, built the data ingestion pipeline, the preprocessing pipeline in Section 3.2, the training augmentation pipeline, and the character vocabulary construction process. Architectural decisions and the final Gradio deployment were a joint effort across all three authors.

5. Experiments and Results

5.1 CRNN Training Convergence

Figure 7 shows CTC training loss with validation CER and WER over 35 epochs. Best validation performance, 5.88 percent CER and 21.15 percent WER, was reached at epoch 30. Training continued five further epochs, and periodic learning rate restarts around epochs 7, 15, and 31 temporarily pushed validation error up before recovery, an expected side effect of Cosine Annealing Warm Restarts. Because the training loop checkpoints the latest epoch rather than the best one, the model actually saved and carried into evaluation reflects epoch 35 (6.78 percent CER, 23.96 percent WER), not the better epoch 30 checkpoint.

We note this plainly as a limitation of the training setup rather than the architecture; saving the best validation checkpoint instead of the latest is a straightforward fix we did not have time to apply and re-run before this report.

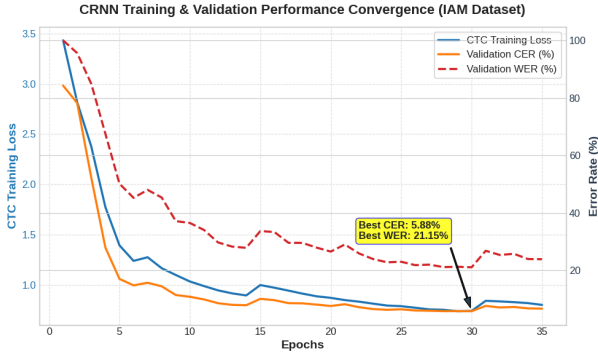


Figure 7. CRNN training loss and validation error across 35 epochs.

5.2 Test Set Results

All results in this section are on the held-out test partition (2,915 lines), unseen during training or validation.

Decoding Strategy	CER (%)	WER (%)
Greedy decoding	9.84	30.86
CTC beam search	9.70	30.45
Beam search + spellcheck	9.13	23.66

Table 1. Pathway A test results by decoding strategy.

Table 1 shows a clear, cumulative improvement from beam search and then spellcheck on top of greedy decoding. Beam search alone gives a small gain; spellcheck on top of it gives a substantially larger one, reducing WER by 7.2 points relative to greedy decoding with no additional training.

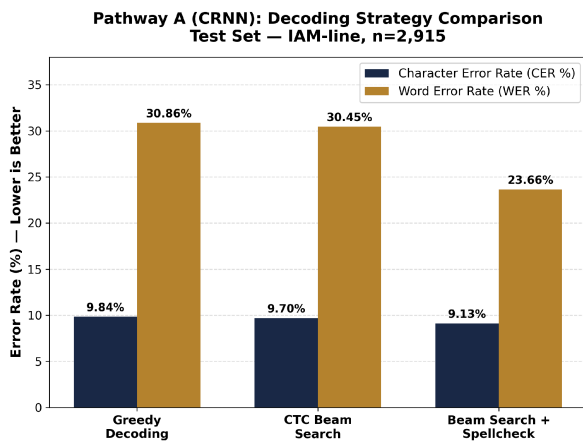


Figure 8. Impact of decoding strategy on CRNN test error rate.

Method	CER (%)	WER (%)
Pathway A: CRNN, beam + spellcheck	9.13	23.66
Pathway B: ViT/HTR-VT, EMA weights	10.62	32.06
Pathway B: ViT/HTR-VT, live weights	11.52	34.14
HTR-VT [7], paper-reported test result	4.70	14.90

Table 2. Final comparison on the IAM-line test set, with the HTR-VT paper's own reported result for context.

On this test set, our CRNN outperforms our integration of the ViT/HTR-VT pathway on both metrics, and both fall short of the result the HTR-VT paper itself reports. We want to be direct about how to read this gap: it is not evidence that the transformer architecture is inherently worse for this task, since the paper's own published result, using the same architecture, is the strongest evidence against that reading. It is more accurately an open question in our specific integration of an externally trained checkpoint. Section 6 discusses the most likely candidates we have not yet fully isolated: truncation of long lines at our fixed input width and remaining preprocessing differences beyond the ink polarity issue we already found and fixed.

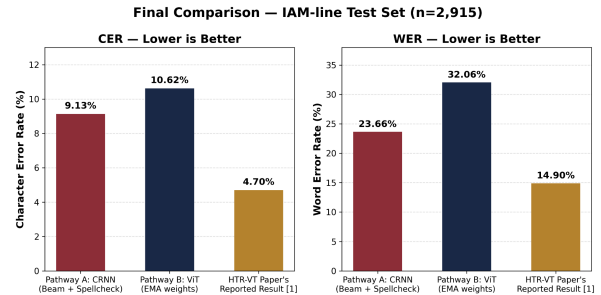


Figure 9. Final CER and WER comparison across both pathways and the published HTR-VT benchmark.

5.3 Model Size and Inference Cost

Pathway A has 10.06 million parameters, computed directly from the architecture. Pathway B, per its training log, has 31.39 million.

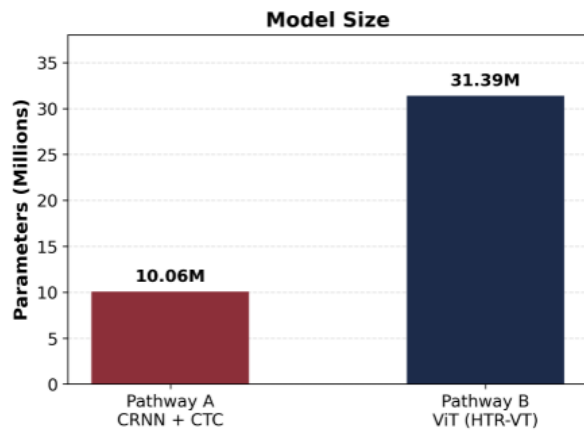


Figure 10. Parameter count comparison between pathways.

We benchmarked post-training INT8 dynamic quantization on Pathway A only, since Pathway B was trained externally and not put through the same step here.

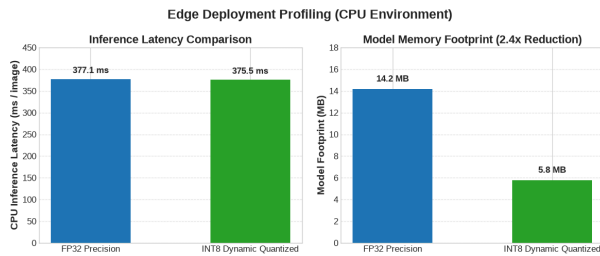


Figure 11. CPU latency and memory footprint before and after INT8 quantization, Pathway A.

Quantization reduced Pathway A's on-disk memory footprint by a factor of 2.4 (14.2 MB to 5.8 MB) but had almost no effect on CPU latency (377.1 ms to 375.5 ms per image, effectively 1.00x). We think this is because PyTorch's dynamic quantization only touches Linear and LSTM layers, while the unquantized convolutional backbone most likely dominates compute time. This is a useful result on its own: quantization is worth doing for deployment size but should not be expected to meaningfully speed up this architecture on CPU without also addressing the convolutional layers.

5.4 Qualitative Analysis

Table 3 shows two representative test examples comparing greedy decoding against beam search with spellcheck. The pattern matches the aggregate numbers: greedy decoding tends to produce plausible but incorrect substitutions (thinn for think, dianges for changes), and spellcheck resolves many, though not all, of these, since it operates at the word level and cannot fix an error that happens to still form a different valid word (movie for Marie, bad for but).

Target	Greedy	Beam + Spellcheck
assuredness " Bella Bella Marie " (Parlophone) , a lively song that changes tempo mid-way .	assuredhness " Bella Bella Mavie " (larlaphone) , a lively song that dianges tempo midway .	assuredness " Bella Bella movie " (larlaphone) , a lively song that dinges tempo midway .
I don't think he will storm the charts with this one , but it 's a good start .	I dont thinn he will storm the charts with this one , bal it 's a good start .	I don't think he will storm the charts with this one , bad it 's a good start .

Table 3. Qualitative test set predictions, Pathway A: greedy decoding versus beam search with spellcheck.

Qualitative Decoding Output Sample Comparison

Pipeline Phase	Transcription Output	Status
Target Label	It was a splendid interpretation of the	Exact Match
Greedy Output	It was a splendid interpretation of that	1 Word Error
Beam + Spellcheck	It was a splendid interpretation of the	Corrected (100%)

Figure 12. A fully corrected example, where spellcheck resolves the sole error a greedy decode produced.

6. Conclusion

This project set out to build and fairly compare two architectures for offline handwriting recognition, and we believe the comparison is fair: both pathways train on the same data, share the same evaluation protocol, and both use CTC loss rather than one using an autoregressive decoder the other does not. Figure 13 summarizes the

debugging journey behind both pathways, from an early, more fundamental problem to a deployed system.

6.1 Key Insights

- The single highest-leverage fix in this project was correcting the CTC target lengths argument from a hardcoded constant of one to the true, per-sample label length. Before this fix, no amount of additional training data or time could have produced multi-character output, because the loss function itself mathematically constrained every prediction to a single character regardless of what the model learned.
- Decoding strategy matters as much as some architectural choices. Moving from greedy decoding to beam search with spellcheck reduced word error rate by 7.2 points on the CRNN pathway with no additional training cost.

- Integrating an externally trained model safely requires verifying assumptions empirically rather than assuming a checkpoint's architecture or input convention matches your own. Two bugs we found in Pathway B, the reconstructed architecture's stride schedule and the ink polarity convention, would have silently produced wrong output that still looked plausible enough to be mistaken for a weak model rather than a wrong assumption, had we not checked them directly against real predictions.

6.2 Limitations

- Our integration of the ViT/HTR-VT pathway (10.62 percent CER) has not closed the gap to the architecture's published result (4.70 percent CER). We consider two candidate causes most likely: truncation of long lines at our fixed input width, since the model's position embeddings are locked to a fixed token count, and remaining preprocessing differences beyond the ink polarity issue already fixed. We could not fully isolate which matters most before this report was due, and this pathway was also not benchmarked for inference latency or quantized, so Section 5.3's efficiency comparison is necessarily one-sided.
- The CRNN's final saved checkpoint (epoch 35) is not its best-performing checkpoint (epoch 30), since the training loop saves the latest epoch rather than the lowest validation error. All CRNN test results in this report reflect the epoch 35 checkpoint.
- The original encoder-decoder transformer design was abandoned rather than fixed, and its failure mode, decoder collapse from cold-start training without pretrained language weights, would likely recur in any future attempt without addressing the root cause directly, for instance by initializing the decoder from pretrained weights.
- Both pathways train exclusively on natural pen-and-paper handwriting from IAM. Neither generalizes well to stylized fonts or digitally drawn text, expected given the training distribution but worth stating plainly as a boundary of what this system was built to do.
- Run a formal, matched-budget benchmark of both pathways including inference latency and quantization for Pathway B.
- Build a character-level confusion analysis identifying which characters are most often confused with which others, beyond what aggregate CER and WER show.

References

- [1] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in Proc. 23rd Int. Conf. Machine Learning, 2006, pp. 369-376.
- [2] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 39, no. 11, pp. 2298-2304, 2017.
- [3] U.-V. Marti and H. Bunke, "The IAM-database: An English sentence database for offline handwriting recognition," Int. J. Document Analysis and Recognition, vol. 5, no. 1, pp. 39-46, 2002.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, 2017.
- [5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in Int. Conf. Learning Representations, 2021.
- [6] M. Li, T. Lv, J. Chu, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in Proc. AAAI Conf. Artificial Intelligence, 2023.
- [7] Y. Li, D. Chen, T. Tang, and X. Shen, "HTR-VT: Handwritten text recognition with vision transformer," Pattern Recognition, vol. 158, 2024.
- [8] Teklia, "IAM-line," Hugging Face Datasets. [Online]. Available: <https://huggingface.co/datasets/Teklia/IAM-line>

6.3 Future Work

- Systematically isolate the remaining ViT accuracy gap through controlled ablation, testing long-line truncation and preprocessing differences independently, and change the training loop to save the best validation checkpoint rather than the latest.
- Expand training data with synthetic and stylized text to broaden generalization beyond natural handwriting.

Supplementary Material

S.1 Informal Qualitative Comparison

During development, both pathways were also tested informally on a small set of handwritten images outside the IAM dataset, collected to sanity check behavior rather than to serve as a rigorous benchmark. Exact ground truth for these images was not systematically recorded alongside predictions, so we report this only as anecdotal evidence, not a quantified result. Both pathways produced noticeably more garbled output than on IAM test images, consistent with the domain-specific limitation noted in Section 6.2.

S.2 Additional Implementation Notes

- Per-epoch checkpoint saves and resumes logic for Pathway A training, so an interrupted run can continue from its last completed epoch.
- An upfront validation pass that filters out corrupt or unreadable image files before training begins, avoiding a mid-epoch crash partway through a long run.
- A full-page line segmentation pipeline, described in Section 4.4, that lets the deployed system accept an arbitrary photo of a note or page rather than requiring pre-cropped lines.